

Frontier-Driven Cooperative Reduction: A Novel GPU Algorithm

By George Chan, 2025

Introduction

In GPU programming, reduction algorithms are fundamental for tasks such as finding maxima, minima, or aggregating values across threads. Traditionally, these reductions rely on warp-level primitives or hierarchical shared memory structures. However, the design of a shared frontier variable, referred to as `iter`, introduces a novel approach that blends reduction with cooperative frontier advancement.

Standard Reduction Patterns

Warp shuffle reduction: Uses warp intrinsics to exchange values between threads, minimizing shared memory usage.

Shared memory reduction: Threads write partial results into shared memory, synchronizing at each step.

Atomic operations: Threads update a global variable directly, relying on atomicity for correctness.

These methods are well-established, but they often involve redundant checks or heavy synchronization.

The Shared Frontier Concept

The frontier-driven approach introduces a shared variable `iter` that represents the lowest index still requiring evaluation. Each thread maintains a local counter (`local_iter`) and compares it against the shared frontier:

If `local_iter < iter`, the thread catches up by setting `local_iter = iter`.

If `local_iter > iter`, the thread advances the frontier by updating `iter = local_iter`.

This mechanism ensures that threads do not waste cycles re-checking positions already surpassed by the frontier.

Algorithm Workflow

Initialization: Shared variables `bestRatio`, `bestIdx`, and `iter` are set.

Loop Execution: Each thread iterates through the input vector:

Synchronizes `local_iter` with `iter`.

Compares local candidate against `bestRatio`.

Updates bestRatio and bestIdx if a better candidate is found.

Advances local_iter and potentially updates iter.

Finalization: After convergence, one thread commits the result to the output.

Advantages

Bug-free correctness: Monotonic updates guarantee convergence without complex locking.

Efficiency: Threads skip redundant checks by adopting the shared frontier.

Simplicity: Requires only one extra shared variable beyond the standard reduction state.

Comparison to Standard Algorithms

Approach

Standard?

Sync style

Pros

Cons

Warp shuffle reduction

Yes

Warp intrinsics

Fast, minimal shared memory

Requires warp-level ops

Shared memory reduction

Yes

Barriers per step

Simple, widely used

More barriers, slower

Atomic global operations

Yes

Lock-free updates

Easy to code, monotonic

High contention, slower

Shared frontier reduction

No

Init/final barrier

Skips redundant checks, simple

Not standard, possible contention

Novelty and Applications

This algorithm is novel because it merges frontier advancement (common in BFS and compaction) with reduction. It is particularly useful in scenarios where:

Input vectors are large and redundant checks are costly.

Deterministic, bug-free convergence is required.

Simplicity is valued over warp-level optimization.

Conclusion

The frontier-driven cooperative reduction algorithm represents an innovative variant in GPU programming. By introducing a shared frontier variable (*iter*), it reduces redundant work while maintaining correctness and simplicity. Though not a standard reduction method, it offers a compelling alternative that blends established paradigms in a novel way.